

« Une entreprise qui ne comprend pas ses données
prend des décisions au hasard avec de la conviction. »

CHAPITRE I – LES DONNÉES : NATURE ET RÉCOLTE

— LES TYPES DE DONNÉES —

Comprendre ce qu'on mesure avant de mesurer est la règle fondamentale. Toutes les données ne se traitent pas de la même façon.

Les données quantitatives continues sont des mesures qui peuvent prendre n'importe quelle valeur dans un intervalle : un chiffre d'affaires, une température, un temps de traitement. On peut calculer une moyenne, un écart-type, une médiane.

Les données quantitatives discrètes sont des nombres entiers qui ne peuvent prendre que des valeurs séparées : nombre de commandes, nombre de clients, nombre d'incidents. On les traite comme des continues si le volume est suffisant.

Les données ordinales sont des catégories avec un ordre logique mais sans distance mesurable entre elles : une note de 1 à 5, un niveau de satisfaction, une priorité haute/moyenne/basse. On peut les classer, pas les additionner.

Les données nominales sont des catégories sans ordre : une ville, une couleur, un secteur d'activité. On les compte, on les groupe, on ne les moyenne jamais.

Les données temporelles sont liées à un moment précis. Elles ont leurs propres méthodes d'analyse. Le temps est toujours une dimension à part dans l'analyse.

Les données textuelles nécessitent un traitement spécifique avant d'être analysables : avis clients, emails, commentaires. On les transforme en données structurées avant de les analyser.

— LES SOURCES DE DONNÉES —

Les données internes sont les plus fiables et les plus riches. Elles viennent des systèmes de l'entreprise : CRM, ERP, outils de facturation, bases de données transactionnelles. Elles décrivent exactement ce que l'entreprise a vécu.

Les données externes enrichissent les données internes. Données sectorielles, benchmarks de marché, données macroéconomiques, données géographiques. Elles donnent le contexte sans lequel les données internes manquent de sens.

Les données d'enquête sont collectées directement auprès de personnes. Leur qualité dépend entièrement de la qualité de la question posée. Une question mal formulée produit une donnée inutilisable, même si elle est précise.

Les données de web analytics mesurent les comportements en ligne : pages visitées, durée de session, taux de rebond, tunnels de conversion. Elles sont volumineuses et bruyantes. Il faut savoir les filtrer.

Les données issues d'APIs donnent accès à des flux en temps réel ou périodiques : réseaux sociaux, places de marché, partenaires. Leur format change sans prévenir. Surveillance

toujours la version de l'API que tu consommes.

Les données issues du scraping sont extraites de sources publiques quand aucune API n'existe. Elles sont fragiles par nature : le DOM d'une page change, le script casse. Ne fonde jamais une décision critique sur une seule source scrapée.

— MÉTHODES DE RÉCOLTE AVEC PANDAS

Pandas est la bibliothèque centrale de toute analyse en Python. Elle transforme n'importe quelle source en DataFrame – une structure de table sur laquelle toutes les opérations d'analyse sont possibles.

Lire un CSV : `pd.read_csv("fichier.csv", encoding="utf-8", sep=";", parse_dates=["date"])`.
Toujours spécifier l'encodage. Toujours parser les dates à l'import plutôt qu'après.

Lire un Excel : `pd.read_excel("fichier.xlsx", sheet_name="Feuille1", skiprows=0)`.
Précise toujours le nom de la feuille. Les fichiers Excel construits par des humains ont souvent des lignes parasites en tête.

Lire depuis une base SQL : `pd.read_sql("SELECT * FROM ventes WHERE date >= '2024-01-01'", con=engine)`.
La requête SQL filtre et agrège avant le chargement. Ne charge jamais toute une table quand tu n'as besoin que d'une partie.

Lire un JSON : `pd.read_json("données.json")` ou `pd.json_normalize(data)` pour aplatir des structures imbriquées.

Lire depuis une API : appel avec `requests`, puis `pd.DataFrame(response.json())` ou `pd.json_normalize()` selon la structure retournée.

— QUALITÉ ET VALIDATION À L'IMPORT

La première chose à faire après avoir chargé des données, c'est de ne pas leur faire confiance. Inspecte avant d'analyser.

`df.shape` donne les dimensions. `df.info()` révèle les types réels et le taux de remplissage. `df.describe()` donne les statistiques de base. Ces trois appels sont le minimum absolu avant toute analyse.

`df.isnull().sum()` compte les valeurs manquantes par colonne. `df.isnull().mean() * 100` donne le pourcentage. Une colonne avec plus de 50% de valeurs manquantes est rarement utilisable sans traitement préalable.

`df.duplicated().sum()` compte les doublons exacts. `df.duplicated(subset=["id"]).sum()` compte les doublons sur la clé fonctionnelle. Ces deux chiffres ne sont pas les mêmes et ne se traitent pas de la même façon.

`df["colonne"].nunique()` donne le nombre de valeurs distinctes. `df["colonne"].value_counts()` montre leur distribution. Toujours regarder la distribution des catégorielles avant d'analyser.

CHAPITRE II – NETTOYAGE ET PRÉPARATION

— VALEURS MANQUANTES

Une valeur manquante peut signifier trois choses différentes : la valeur est inconnue, la valeur est zéro, ou la valeur ne s'applique pas. Ces trois cas ne se traitent pas de la même façon. Valide toujours avec le métier avant de décider.

Supprimer les lignes dont la colonne clé est manquante :
`df.dropna(subset=["id", "date"])`.

Remplacer par zéro quand l'absence signifie zéro :
`df["montant"].fillna(0)`.

Remplacer par la médiane sur des données numériques asymétriques,
par la moyenne sur des données symétriques :
`df["prix"].fillna(df["prix"].median())`.

Créer une catégorie "INCONNU" pour les nominales :
`df["segment"].fillna("INCONNU")`.

Ne jamais remplacer mécaniquement sans comprendre ce que
signifie l'absence. C'est la décision de nettoyage la plus
importante de toute l'analyse.

— TYPES ET FORMATS

Les dates mal parsées sont la source d'erreur la plus fréquente
en data. `pd.to_datetime(df["date"], format="%Y-%m-%d",
errors="coerce")` convertit et met NaT là où la conversion
échoue. Compte toujours les NaT produits.

Les numériques stockés en texte arrivent souvent avec des
caractères parasites : symboles monétaires, espaces, virgules
décimales. Nettoie avant de convertir :
`df["montant"].str.replace("€","").str.replace(",",".").str.strip()`
puis `pd.to_numeric(df["montant"], errors="coerce")`.

Les catégorielles ont souvent des variantes orthographiques.
`str.strip()` supprime les espaces. `str.upper()` ou `str.lower()`
harmonise la casse. Un dictionnaire de mapping corrige les
variantes connues.

Convertir en type `category` économise la mémoire et accélère
les opérations de groupe sur les catégorielles :
`df["statut"] = df["statut"].astype("category")`.

— OUTLIERS

Un outlier est une valeur qui s'écarte fortement du reste.
Il peut être une erreur de saisie, une fraude, ou une réalité
exceptionnelle. Ne jamais supprimer sans comprendre lequel.

La méthode IQR est la plus robuste. On calcule Q1 et Q3 avec
`df["col"].quantile(0.25)` et `df["col"].quantile(0.75)`. L'IQR
est $Q3 - Q1$. Les bornes sont $Q1 - 1.5 \times IQR$ et $Q3 + 1.5 \times IQR$.
Ce qui dépasse les bornes est un outlier potentiel.

Le z-score avec `scipy.stats.zscore(df["col"])` donne le nombre
d'écarts-types de chaque valeur par rapport à la moyenne.
Un z-score supérieur à 3 en valeur absolue signale un outlier
probable. Cette méthode est sensible aux données asymétriques.

La bonne pratique est de créer un flag plutôt que de supprimer :
`df["is_outlier"] = (df["col"] < borne_basse) |
 (df["col"] > borne_haute).`
Le client décide de l'action. L'analyste signale.

CHAPITRE III – STATISTIQUES DESCRIPTIVES

— MESURES DE TENDANCE CENTRALE

Les statistiques descriptives répondent à la question : que
s'est-il passé ? Elles résument, sans extrapoler.

La moyenne est la somme divisée par le nombre d'observations.

Elle est influencée par les valeurs extrêmes. Sur des données de revenus ou de prix, une poignée de valeurs élevées tire la moyenne vers le haut et donne une image fausse du "typique".

La médiane est la valeur centrale qui partage les données en deux moitiés égales. Elle est robuste aux valeurs extrêmes. Sur des données asymétriques – salaires, prix immobiliers, chiffres d'affaires – la médiane est presque toujours plus représentative que la moyenne.

Le mode est la valeur la plus fréquente. Il est utile sur les catégorielles et les discrètes. Sur les continues, il perd de son sens sauf si la distribution est clairement bimodale.

En pratique : si moyenne et médiane sont proches, la distribution est symétrique. Si elles divergent, la distribution est asymétrique et la médiane est ton indicateur de référence.

Avec numpy : `np.mean()`, `np.median()`, `scipy.stats.mode()`.
Avec pandas : `df["col"].mean()`, `df["col"].median()`, `df["col"].mode()[0]`.

— MESURES DE DISPERSION

La dispersion dit à quel point les données sont concentrées ou éparpillées autour de la tendance centrale. Deux populations avec la même moyenne peuvent être radicalement différentes dans leur dispersion.

L'écart-type est la racine carrée de la variance. Il exprime la dispersion dans la même unité que la donnée. Un écart-type élevé par rapport à la moyenne indique une forte hétérogénéité. `np.std(df["col"], ddof=1)` pour l'écart-type d'un échantillon.

La variance est l'écart-type au carré. Elle est moins intuitive car son unité est le carré de celle de la donnée. On l'utilise dans les calculs statistiques, pas dans la communication des résultats.

L'intervalle interquartile ($IQR = Q3 - Q1$) est la dispersion des 50% centraux. Il est insensible aux valeurs extrêmes. C'est la mesure de dispersion à privilégier quand les données sont asymétriques.

Le coefficient de variation $CV = \text{écart-type} / \text{moyenne}$ permet de comparer la dispersion entre des variables d'unités différentes. Un CV de 0.1 signifie que l'écart-type est 10% de la moyenne – distribution homogène. Un CV de 0.8 signifie une forte hétérogénéité.

— DISTRIBUTION ET FORME

La skewness mesure l'asymétrie d'une distribution. `scipy.stats.skew(df["col"])` retourne un coefficient. Une valeur proche de 0 indique une distribution symétrique. Une valeur positive indique une queue à droite – beaucoup de petites valeurs et quelques très grandes. Une valeur négative indique une queue à gauche.

Les données de revenus, de prix et de temps d'attente sont presque toujours asymétriques à droite. C'est la norme, pas l'exception.

La kurtosis mesure l'aplatissement. `scipy.stats.kurtosis()` retourne un coefficient. Une kurtosis élevée indique une distribution à queues épaisses – les valeurs extrêmes sont plus fréquentes qu'une distribution normale.

Les percentiles donnent la distribution complète. `df["col"].quantile([0.05, 0.25, 0.5, 0.75, 0.95])` montre les seuils à 5%, 25%, 50%, 75% et 95% de la population. C'est souvent plus utile que la moyenne pour décrire une distribution à un décideur.

CORRÉLATIONS

La corrélation mesure la force et la direction de la relation linéaire entre deux variables. Elle varie entre -1 et +1.

Le coefficient de Pearson est le plus courant.
`df[["col1", "col2"]].corr()` donne la matrice complète.
numpy : `np.corrcoef(x, y)`. Une corrélation de 0.8 est forte, 0.5 est modérée, 0.2 est faible. Ces seuils varient selon le domaine et le contexte.

Le coefficient de Spearman est plus robuste sur les données asymétriques et les relations non linéaires.
`scipy.stats.spearmanr(x, y)` retourne le coefficient et la p-value. À privilégier quand les données ne sont pas normalement distribuées.

La règle absolue : corrélation ne signifie pas causalité. Deux variables peuvent être fortement corrélées parce qu'elles partagent une cause commune, parce que l'une cause l'autre, ou simplement par coïncidence sur l'échantillon observé. Ne jamais présenter une corrélation comme une explication causale sans analyse complémentaire.

AGRÉGATIONS ET TABLEAUX CROISÉS

L'agrégation est le cœur de l'analyse descriptive. Elle transforme des millions de lignes en quelques chiffres compréhensibles et actionnables.

```
df.groupby("segment").agg(  
    nb_clients      = ("id",      "count"),  
    ca_total        = ("montant", "sum"),  
    ca_moyen        = ("montant", "mean"),  
    ca_median       = ("montant", "median"),  
    premier_achat   = ("date",    "min"),  
    dernier_achat   = ("date",    "max")  
) .reset_index()
```

`pd.crosstab(df["region"], df["segment"], values=df["montant"], aggfunc="sum", normalize="index")` donne la répartition du chiffre d'affaires par segment au sein de chaque région, en pourcentage.

La règle des 80/20 se vérifie sur presque toutes les données d'entreprise. 20% des clients génèrent 80% du CA. 20% des produits génèrent 80% des ventes. La quantifier précisément est toujours un résultat parlant pour un décideur.

CHAPITRE IV – STATISTIQUES INFÉRENTIELLES

PRINCIPE FONDAMENTAL

Les statistiques inférentielles répondent à la question : ce qu'on observe dans l'échantillon est-il vrai dans la population entière, ou est-ce dû au hasard ?

On ne peut jamais être certain. On peut seulement quantifier le risque de se tromper. C'est le rôle de la p-value et des intervalles de confiance.

L'hypothèse nulle H_0 est l'hypothèse du "rien ne se passe" : les deux groupes sont identiques, il n'y a pas d'effet, il n'y a pas de relation. L'hypothèse alternative H_1 est ce qu'on cherche à démontrer.

La p-value est la probabilité d'observer un résultat au moins aussi extrême que celui observé, si H_0 était vraie. Une p-value

de 0.03 signifie qu'il y aurait 3% de chances d'observer ça par hasard. Si ce seuil est inférieur à alpha (généralement 0.05), on rejette H_0 .

Rejeter H_0 ne prouve pas H_1 . Cela dit seulement que les données sont incompatibles avec H_0 au niveau de confiance choisi. La nuance est importante, surtout en communication client.

— TESTS SUR LES MOYENNES —

Le test t de Student compare les moyennes de deux groupes. `scipy.stats.ttest_ind(groupe_A, groupe_B)` pour deux groupes indépendants. `scipy.stats.ttest_rel(avant, après)` pour des mesures répétées sur le même groupe.

Il retourne la statistique t et la p-value. Si $p < 0.05$, la différence de moyennes est statistiquement significative – elle n'est probablement pas due au hasard.

Cas d'usage stratégique : comparer le panier moyen avant et après une campagne. Comparer la satisfaction client entre deux agences. Comparer le temps de traitement entre deux processus.

L'ANOVA (analyse de variance) compare les moyennes de plus de deux groupes. `scipy.stats.f_oneway(groupe_A, groupe_B, groupe_C)` teste si au moins un groupe a une moyenne significativement différente des autres.

Cas d'usage : comparer les performances de plusieurs régions, de plusieurs équipes commerciales, de plusieurs gammes de produits.

— TESTS SUR LES PROPORTIONS ET LES FRÉQUENCES —

Le test du chi-deux teste si deux variables catégorielles sont indépendantes. `scipy.stats.chi2_contingency(tableau)` retourne la statistique, la p-value, les degrés de liberté et les fréquences attendues.

Cas d'usage : est-ce que le taux de conversion dépend du canal d'acquisition ? Est-ce que la satisfaction dépend du segment client ? Est-ce que le taux de churn dépend de la région ?

Le test de proportion `scipy.stats.proportions_ztest([n_succès_A, n_succès_B], [n_total_A, n_total_B])` compare deux taux.

Cas d'usage : comparer les taux de conversion de deux versions d'un processus. Comparer les taux d'incidents de deux fournisseurs.

— INTERVALLES DE CONFIANCE —

Un intervalle de confiance à 95% dit : si on répétait cette étude 100 fois, dans 95 cas l'intervalle calculé contiendrait la vraie valeur de la population.

`scipy.stats.t.interval(0.95, df=n-1, loc=moyenne, scale=sem)` calcule l'intervalle de confiance à 95% pour une moyenne. `sem` est l'erreur standard : `scipy.stats.sem(données)`.

Communiquer un intervalle de confiance plutôt qu'un chiffre seul est plus honnête et plus utile. "Le CA moyen par client est de 1 250€ avec un intervalle de confiance à 95% entre 1 180€ et 1 320€" permet une décision éclairée. Un chiffre seul donne une fausse précision.

— CORRÉLATION ET RÉGRESSION —

La régression linéaire simple modélise la relation entre une variable explicative X et une variable cible Y.

`scipy.stats.linregress(x, y)` retourne la pente, l'intercept, le coefficient de corrélation, la p-value et l'erreur standard.

La pente dit de combien Y varie quand X augmente d'une unité. R^2 dit quelle proportion de la variance de Y est expliquée par X. Un R^2 de 0.7 signifie que X explique 70% de la variation de Y.

Cas d'usage : la satisfaction client explique-t-elle la fidélisation ? Le budget publicitaire explique-t-il le CA ? Le délai de livraison explique-t-il le taux de retour ?

La régression ne prouve pas la causalité. Elle quantifie une association. La causalité se démontre par le design de l'étude, pas par le modèle statistique.

CHAPITRE V – ANALYSE TEMPORELLE

— PRÉPARER LES DONNÉES TEMPORELLES —

Le temps est une dimension à part. Les données temporelles ont des propriétés que les autres n'ont pas : saisonnalité, tendance, autocorrélation. Ces propriétés changent les méthodes et les interprétations.

Convertir en datetime est le prérequis de tout.
`df["date"] = pd.to_datetime(df["date"])`. Puis indexer sur la date : `df = df.set_index("date")`. Un DataFrame avec un index datetime déverrouille toutes les opérations temporelles de pandas.

Créer les colonnes dérivées utiles à l'analyse :
`df["annee"], df["mois"], df["semaine"], df["trimestre"], df["jour_semaine"]` à partir de `df.index.year`, `df.index.month`, `df.index.isocalendar().week`, `df.index.quarter`, `df.index.day_name()`.

Resampler agrège par période temporelle. `df["montant"].resample("M").sum()` donne le total mensuel. `resample("Q").mean()` donne la moyenne trimestrielle. `resample("W").count()` donne le nombre hebdomadaire. C'est l'opération centrale de l'analyse de séries.

— TENDANCE —

La tendance est la direction générale sur le long terme. Elle élimine le bruit et la saisonnalité pour révéler si l'indicateur monte, descend ou stagne structurellement.

La moyenne mobile lisse la série et fait apparaître la tendance.
`df["mm_3m"] = df["montant"].rolling(window=3).mean()` calcule la moyenne sur les 3 dernières périodes. `df["mm_12m"] = df["montant"].rolling(window=12).mean()` sur les 12 dernières. Plus la fenêtre est large, plus la courbe est lisse, plus la tendance est visible.

La moyenne mobile exponentielle donne plus de poids aux observations récentes. `df["mme"] = df["montant"].ewm(span=3, adjust=False).mean()`. Elle réagit plus vite aux changements de direction.

Le taux de croissance période sur période est l'indicateur le plus parlant pour un décideur. `df["croissance"] = df["montant"].pct_change() * 100` donne la variation en pourcentage par rapport à la période précédente.

La croissance glissante sur 12 mois élimine la saisonnalité dans la mesure de la tendance. `df["croissance_N-1"] = df["montant"].pct_change(12) * 100`.

— SAISONNALITÉ —

La saisonnalité est un pattern qui se répète à intervalles réguliers. Elle est structurelle – elle existe indépendamment de la tendance et du bruit.

Identifier la saisonnalité : grouper par période et comparer.
`df.groupby(df.index.month)["montant"].mean()` donne le CA moyen par mois de l'année. Si décembre est systématiquement le double de janvier, la saisonnalité est forte.

`df.groupby(df.index.day_name())["montant"].mean()` donne la distribution par jour de la semaine. Utile pour les activités à forte variation journalière.

L'indice de saisonnalité compare chaque mois à la moyenne annuelle. Un indice de 1.4 pour décembre signifie que décembre dépasse de 40% la moyenne mensuelle. Ces indices permettent de faire des projections ajustées de la saisonnalité.

Tenir compte de la saisonnalité avant de tirer des conclusions. Une baisse de CA en janvier après décembre n'est souvent pas une mauvaise nouvelle – c'est simplement la saisonnalité. Comparer à janvier N-1 est plus juste que comparer à décembre.

— DÉTECTION D'ANOMALIES TEMPORELLES —

Une anomalie dans une série temporelle est un point qui s'écarte significativement du comportement attendu compte tenu de la tendance et de la saisonnalité.

La méthode par écart-type : calculer la moyenne et l'écart-type sur une fenêtre glissante. Un point qui dépasse moyenne ± 2 écarts-types sur cette fenêtre est suspect.

```
rolling_mean = df["montant"].rolling(window=30).mean()
rolling_std = df["montant"].rolling(window=30).std()
df["anomalie"] = (df["montant"] > rolling_mean + 2*rolling_std)
                | (df["montant"] < rolling_mean - 2*rolling_std)
```

Les anomalies sont souvent les observations les plus intéressantes. Elles signalent un événement exceptionnel : une campagne, une panne, une fraude, un bug de collecte. Ne jamais les supprimer sans comprendre leur cause.

— COMPARAISONS TEMPORELLES POUR LA DÉCISION —

La comparaison N vs N-1 est la plus intuitive. Elle élimine la saisonnalité naturellement en comparant des périodes équivalentes. C'est le point de départ de tout rapport de performance.

Le YTD (Year To Date) compare les cumuls depuis le début de l'année. `df[df.index.year == 2024]["montant"].sum()` vs `df[df.index.year == 2023]["montant"].sum()` sur la même période.

La période glissante sur 12 mois lisse les effets de saisonnalité et donne une vision de la tendance structurelle. `df["montant"].rolling(window=12).sum()` sur des données mensuelles donne le CA des 12 derniers mois à chaque point.

Le CAGR (Compound Annual Growth Rate) mesure le taux de croissance annuel moyen sur une période.
$$\text{CAGR} = (\text{valeur_finale} / \text{valeur_initiale})^{(1/\text{nb_années})} - 1.$$
C'est l'indicateur de tendance long terme le plus utilisé en contexte stratégique.

Une visualisation pose une question. Si tu ne sais pas quelle question elle pose, n'envoie pas ce graphique.

Le titre est une conclusion, pas une description. "Les ventes ont reculé de 18% en Q3 malgré une hausse du trafic" est un titre qui déclenche une réflexion. "Ventes Q3 par canal" est un titre qui ne dit rien.

Une couleur principale, une couleur d'accent. Tout ce qui n'est pas le message principal passe en gris. L'œil va naturellement vers la couleur. Dirige-le vers ce qui compte.

Supprime tout ce qui ne porte pas d'information. Légendes inutiles, grilles surchargées, axes en double, titres d'axes évidents. Chaque élément doit gagner sa place.

— QUEL GRAPHIQUE POUR QUELLE QUESTION —

Pour montrer une évolution dans le temps, utilise une courbe. `ax.plot(x, y)`. Ajoute une zone ombrée avec `ax.fill_between` pour renforcer l'impression de tendance. Annote les points remarquables directement sur le graphique.

Pour comparer des catégories entre elles, utilise des barres horizontales quand les étiquettes sont longues, verticales quand elles sont courtes. `ax.barh()` ou `ax.bar()`. Classe toujours du plus grand au plus petit. Annote les valeurs directement sur les barres avec `ax.bar_label()`.

Pour montrer une distribution, utilise un histogramme. `ax.hist(df["col"], bins=30, edgecolor="white")`. Ajoute une ligne verticale pour la médiane avec `ax.axvline()`.

Pour montrer la relation entre deux variables continues, utilise un nuage de points. `ax.scatter(x, y, alpha=0.5)`. Ajoute la droite de régression pour visualiser la corrélation.

Pour montrer la répartition de parties dans un tout, utilise des barres empilées plutôt que des camemberts. Les camemberts sont difficiles à lire dès qu'il y a plus de 3 parties.

Pour montrer une matrice de corrélation, utilise une heatmap avec matplotlib et numpy : `ax.imshow(corr_matrix)` et `ax.set_xticks / ax.set_yticks` pour les étiquettes.

— CONFIGURATION POUR LES LIVRABLES CLIENT —

Configurer matplotlib une fois en début d'analyse :

```
plt.rcParams.update({"figure.figsize": (12, 6),  
"axes.spines.top": False, "axes.spines.right": False,  
"font.family": "sans-serif", "axes.titlesize": 14,  
"axes.titleweight": "bold", "axes.titlepad": 12}).
```

Supprimer les bordures haut et droite donne immédiatement un rendu plus propre et plus professionnel. C'est la modification la plus impactante avec le moins d'effort.

Sauvegarder en PNG 150 dpi minimum pour les rapports :

```
plt.savefig("output/graphique.png", dpi=150, bbox_inches="tight").
```

`bbox_inches="tight"` évite les coupures sur les axes et titres.

Toujours fermer la figure après sauvegarde : `plt.close()`.
En production, les figures non fermées accumulent de la mémoire.

Un rapport d'analyse n'est pas un compte-rendu de tout ce qu'on a fait. C'est une réponse à une question stratégique. Formule la question avant d'ouvrir le fichier.

Connais ton audience. Un rapport pour un directeur financier n'est pas un rapport pour un responsable opérationnel. Le premier veut des chiffres et des risques. Le second veut des causes et des actions. Les deux peuvent lire le même rapport si tu l'as bien structuré.

Distingue les faits des interprétations et les interprétations des recommandations. Ces trois niveaux doivent être clairement séparés dans le rapport. Un décideur qui confond un fait et une interprétation prend de mauvaises décisions.

— STRUCTURE D'UN RAPPORT D'ANALYSE —

L'EXECUTIVE SUMMARY vient en premier et résume tout le rapport en une page. Il contient : le contexte de l'analyse, les trois à cinq findings principaux, et les recommandations qui en découlent. C'est la seule partie que certains décideurs liront. Écris-la en dernier, mais place-la en premier.

LE CONTEXTE explique pourquoi cette analyse a été faite, quelles données ont été utilisées, sur quelle période, et quelles sont leurs limites connues. La transparence sur les limites des données renforce la crédibilité de l'analyse.

LES FINDINGS sont les résultats. Chaque finding est : une observation factuelle appuyée par un ou deux graphiques ou tableaux, suivie de son interprétation dans le contexte métier. Pas plus de cinq à sept findings principaux. Au-delà, les messages se noient.

LES RECOMMANDATIONS découlent directement des findings. Elles sont spécifiques, actionnables, et ordonnées par impact. "Améliorer la satisfaction client" n'est pas une recommandation. "Réduire le délai de traitement des réclamations de 48h à 24h en priorisant les clients segment A" en est une.

LES ANNEXES contiennent les détails méthodologiques, les tableaux complets, et les graphiques supplémentaires. Tout ce qui est nécessaire pour vérifier le travail mais pas pour prendre la décision.

— RÈGLES DE COMMUNICATION DES RÉSULTATS —

Chaque chiffre présenté a besoin d'une référence. Un CA de 2,3M€ – est-ce bien ou mal ? Par rapport à l'objectif, à N-1, au secteur ? Sans référence, un chiffre seul ne déclenche aucune réflexion utile.

Les pourcentages de variation trompent quand la base est petite. Passer de 2 à 4 clients, c'est +100% – ce n'est pas le même phénomène qu'un réseau qui passe de 2 000 à 4 000 clients. Toujours indiquer les valeurs absolues avec les variations.

Sépare ce qui est statistiquement significatif de ce qui ne l'est pas. Présenter une variation de 2% sur un petit échantillon comme une tendance, sans mentionner l'incertitude, c'est tromper le décideur même involontairement.

Indique la source, la période, et la méthode de calcul de chaque indicateur. Une même métrique calculée différemment par deux équipes produit deux chiffres différents et deux décisions contradictoires.

Le rapport doit pouvoir être compris par quelqu'un qui n'était pas dans la salle quand les données ont été analysées. Si ce n'est pas le cas, il ne sera pas utilisé au-delà du premier mois.

L'analyse est au service de la décision. Une analyse qui ne change aucune décision est une analyse inutile, peu importe sa qualité technique.

Avant de livrer, pose-toi la question : si le décideur lit ce rapport, quelle décision différente va-t-il prendre ? Si tu ne peux pas répondre, retravaille les recommandations.

Les décisions stratégiques portent sur l'allocation de ressources, l'entrée ou sortie de marchés, le développement ou l'arrêt de produits, la restructuration d'opérations. L'analyse doit être calibrée au niveau de la décision – ni trop détaillée, ni trop vague.

La limite de l'analyse quantitative est que les données décrivent le passé. La décision concerne le futur. L'analyse réduit l'incertitude, elle ne l'élimine pas. Le rôle de l'analyste est de quantifier ce qu'on sait, de nommer ce qu'on ne sait pas, et de laisser le décideur décider avec les deux en main.

VÉRITÉS DU TERRAIN

Les données décrivent ce qui s'est passé. Elles n'expliquent pas pourquoi. L'explication vient du croisement entre les données et la connaissance du terrain. L'analyste seul n'a que la moitié du tableau.

Un résultat statistiquement significatif n'est pas forcément significatif pour l'entreprise. Une différence de 0.3% entre deux groupes peut être significative avec un million de données sans changer aucune décision concrète.

La qualité de l'analyse est limitée par la qualité des données. Produire un rapport sophistiqué sur des données mal collectées est une erreur qui se voit rarement dans le rapport et toujours dans la réalité.

Un rapport qu'on relit six mois plus tard pour comprendre pourquoi une décision a été prise – c'est un bon rapport. Un rapport qu'on ne retrouve plus, ou qu'on ne comprend plus, c'est du travail perdu.

La data sans storytelling reste dans les slides. Le storytelling sans data reste dans les têtes. L'aide à la décision efficace c'est les deux ensemble, dans le bon ordre.
